Singly Linkedlist

```
#include <iostream>
using namespace std;
struct Node
{
   int memberID;
   Node* next;
};
Node* HEAD = NULL;
void createNode()
{
   int id;
   cout << "Enter Member ID: ";
   cin >> id;
   Node* newNode = new Node;
   newNode->memberID = id;
   newNode->next = NULL;
   if (HEAD == NULL)
   {
      HEAD = newNode;
   }
   else
   {
      Node* temp = HEAD;
      while (temp->next != NULL)
      {
         temp = temp->next;
      }
      temp->next = newNode;
   }
   cout << "Node created successfully.\n";
}
void insertBeginning()
{
   int id;
   cout << "Enter Member ID: ";
   cin >> id;
   Node* newNode = new Node;
   newNode->memberID = id;
   newNode->next = HEAD;
   HEAD = newNode;
   cout << "Member inserted at the beginning successfully.\n";
}
void insertEnd()
{
   int id;
   cout << "Enter Member ID: ";
   cin >> id;
   Node* newNode = new Node;
```

```
    newNode->memberID = id;
    newNode->next = NULL;
    if (HEAD == NULL)
    {
        HEAD = newNode;
    }
    else
    {
        Node* temp = HEAD;
        while (temp->next != NULL)
        {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    cout << "Member inserted at the end successfully.\n";
}
int countNodes()
{
    int count = 0;
    Node* temp = HEAD;
    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }
    return count;
}
void insertPosition()
{
    int id;
    int position;
    cout << "Enter Member ID: ";
    cin >> id;
    cout << "Enter Position: ";
    cin >> position;
    int total = countNodes();
    if (position < 1 || position > total + 1)
    {
        cout << "Invalid position.\n";
        return;
    }
    Node* newNode = new Node;
    newNode->memberID = id;
    if (position == 1)
    {
        newNode->next = HEAD;
        HEAD = newNode;
    }
```

```
    else

    {
        Node* temp = HEAD;
        int counter = 1;
        while (counter < position - 1)
        {
            temp = temp->next;
            counter++;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }
    cout << "Member inserted at position " << position << " successfully.\n";
}
void display()
{
    if (HEAD == NULL)
    {
        cout << "Membership list is empty.\n";
        return;
    }
    Node* temp = HEAD;
    cout << "\nCoding Club Membership Records\n";
    while (temp != NULL)
    {
        cout << "Member ID: " << temp->memberID << endl;
        temp = temp->next;
    }
}
void searchMember()
{
    int id;
    cout << "Enter Member ID to search: ";
    cin >> id;
    if (HEAD == NULL)
    {
        cout << "Membership list is empty.\n";
        return;
    }
    Node* temp = HEAD;
    while (temp != NULL)
    {
        if (temp->memberID == id)
        {
            cout << "\nMember found.\n";
            cout << "Member ID: " << temp->memberID << endl;
            return;
        }
        temp = temp->next;
```

```
    }
    cout << "Member not found.\n";
}
void deleteBeginning()
{
    if (HEAD == NULL)
    {
        cout << "Membership list is empty.\n";
        return;
    }
    Node* temp = HEAD;
    cout << "Deleted Member ID: " << temp->memberID << endl;
    HEAD = HEAD->next;
    delete temp;
    cout << "Member deleted from the beginning successfully.\n";
}
void deleteEnd()
{
    if (HEAD == NULL)
    {
        cout << "Membership list is empty.\n";
        return;
    }
    if (HEAD->next == NULL)
    {
        cout << "Deleted Member ID: " << HEAD->memberID << endl;
        delete HEAD;
        HEAD = NULL;
        cout << "Member deleted from the end successfully.\n";
        return;
    }
    Node* previous = HEAD;
    Node* current = HEAD->next;
    while (current->next != NULL)
    {
        previous = current;
        current = current->next;
    }
    cout << "Deleted Member ID: " << current->memberID << endl;
    previous->next = NULL;
    delete current;
    cout << "Member deleted from the end successfully.\n";
}
void deletePosition()
{
    int position;
    cout << "Enter Position to delete: ";
    cin >> position;
    if (HEAD == NULL)
    {
```

```
        cout << "Membership list is empty.\n";
        return;
    }
    int total = countNodes();
    if (position < 1 || position > total)
    {
        cout << "Invalid position.\n";
        return;
    }
    if (position == 1)
    {
        Node* temp = HEAD;
        cout << "Deleted Member ID: " << temp->memberID << endl;
        HEAD = HEAD->next;
        delete temp;
        cout << "Member deleted successfully.\n";
        return;
    }
    Node* previous = HEAD;
    Node* current = HEAD->next;
    int counter = 2;
    while (counter < position)
    {
        previous = current;
        current = current->next;
        counter++;
    }
    cout << "Deleted Member ID: " << current->memberID << endl;
    previous->next = current->next;
    delete current;
    cout << "Member deleted from position " << position << " successfully.\n";
}
void updateMember()
{
    int id;
    cout << "Enter Member ID to update: ";
    cin >> id;
    if (HEAD == NULL)
    {
        cout << "Membership list is empty.\n";
        return;
    }
    Node* temp = HEAD;
    while (temp != NULL)
    {
        if (temp->memberID == id)
        {
            cout << "Enter Updated Member ID: ";
            cin >> temp->memberID;
            cout << "Member details updated successfully.\n";
```

```
            return;
        }
        temp = temp->next;
    }
    cout << "Member not found.\n";
}
int main()
{
    int choice;
    do
    {
        cout << "CODING CLUB MEMBER MANAGEMENT\n";
        cout << "1. Create Node\n";
        cout << "2. Insert at Beginning\n";
        cout << "3. Insert at End\n";
        cout << "4. Insert at Specific Position\n";
        cout << "5. Display Membership Records\n";
        cout << "6. Search Member\n";
        cout << "7. Delete from Beginning\n";
        cout << "8. Delete from End\n";
        cout << "9. Delete from Specific Position\n";
        cout << "10. Update Member\n";
        cout << "11. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
            case 1:
                createNode();
                break;
            case 2:
                insertBeginning();
                break;
            case 3:
                insertEnd();
                break;
            case 4:
                insertPosition();
                break;
            case 5:
                display();
                break;
            case 6:
                searchMember();
                break;
            case 7:
                deleteBeginning();
                break;
            case 8:
```

```
            deleteEnd();
            break;
        case 9:
            deletePosition();
            break;
        case 10:
            updateMember();
            break;
        case 11:
            cout << "Program terminated.\n";
            break;
        default:
            cout << "Invalid choice. Please try again.\n";
      }
   } while (choice != 11);
   return 0;
}
```